Divisione del Lavoro:
Mattia: prova pratica di MongoDB, DigitalHub
Davide: prova pratica di PostgreSQL, DataVerse
Andrea: prova pratica di MinIO, Apache Iceberg, motore di query, DataHub
Tutti: OpenMetadata, DigitalHub, attività 1.6 e 1.7

Griglia di confronto

Sistema	1. A cosa serve	2. Organizzazione dati e metadati	3. Versionamento	4. Identificazione del dato	5. Facilità di installazione e d'uso	6. Apertura e portabilità
MongoDB	Database NoSQL orientato ai documenti, utilizzato per la memorizzazione e la gestione dei dati. I dati vengono salvati in documenti BSON, organizzati all'interno di collection	Le collection sono l'equivalente delle tabelle relazionali, mentre i documenti rappresentano i singoli record. La differenza fondamentale è che una collection MongoDB può contenere documenti con strutture differenti	Non offre un vero versionamento automatico dei documenti; quando un documento viene modificato, normalmente il suo stato precedente non è recuperabile semplicemente tramite una normale query	Ogni documento possiede normalmente un campo _id, che permette di identificarlo univocamente all'interno della collection. La gerarchia può essere vista come: Database → Collection → Documento → _id	Semplice da utilizzare, è sufficiente avere familiarità con i JSON. È possibile creare una collection ed iniziare immediatamente ad inserire documenti. Installazione rapida tramite docker	Utilizza principalmente BSON, una rappresentazione binaria simile a JSON, per la memorizzazione dei documenti. I dati possono comunque essere esportati in formati più facilmente utilizzabili da altri strumenti, come JSON e CSV, tramite gli strumenti di esportazione di MongoDB
PostgreSQL	Database relazionale che offre la possibilità di salvare tipi di dato differenti, anche in formato JSON, mantenendo una struttura tabellare	Organizzazione in tabelle, ossia un insieme ordinato di attributi, ognuno dei quali varia in un certo dominio. Possibilità di legare dati presenti in altre tabelle tramite chiavi esterne	Non offre nativamente la possibilità di recuperare uno stato passato. Il ripristino è consentito solo se configurato in anticipo	Ogni riga della tabella è unicamente identificata da un camp chiave (Primary Key), pertanto non possono esistere due righe con l stessa chiave	Immediata tramite Docker, poiché richiesto solo il pull e l'avvio di un container. Argomenti avanzati, quali indicizzazione, richiedono maggiore attenzione e conoscenze di strutture dati e algoritmi.	La licenza PostgreSQL è permissiva e approvata OSI. I dati sono esportabili liberamente in formati standard, quali CSV, senza nessun vincolo
Apache Iceberg	Gestione e analisi di dati con storico versionato, per interrogazioni analitiche anche su intervalli molto ampi	Dati in file Parquet immutabili su MinIO; "tabella" logica ricostruita da gerarchia di metadati (manifest file, manifest list, metadata.json) ; catalogo su Postgres = solo puntatore all'ultimo metadata, senza schema né dati	Ogni scrittura genera un nuovo snapshot immutabile; snapshot precedenti interrogabili finché non scadono, verificato con time travel (snapshot_from_id / snapshot_from_timestamp)	Namespace/nome all'interno del catalogo, valore solo interno; citabilità esterna demandata alla pubblicazione su Dataverse	Semplice commando pip install	Apache 2.0; dati (Parquet/JSON) portabili e leggibili indipendentemente dal motore di query usato
DuckDb	Motore SQL analitico *embedded* per interrogare Iceberg in SQL vero e leggerne il piano di esecuzione	Nessuna tabella propria persistente nel prototipo: ogni query è una chiamata a ceberg_scan(metadata_path)	Nessuno proprio: eredita quello di Iceberg tramite i parametri di iceberg_scan	Non applicabile in sens stretto: nessun nome di tabella persistente in DuckDB, solo il percorso del metadata passato a runtime	Installazione immediata (pip), nessun server; non supporta nativamente il catalogo SQL/JDBC scelto (richiede bridge via PyIceberg);	Licenza MIT, SQL standard; estensioni iceberg e httpfs open source
MinIO	Object storage compatibile S3; storage fisico dei file Iceberg	Bucket e oggetti identificati da chiavi; non è un filesystem, nessuna cartella fisica reale	Versionamento nativo disponibile	Oggetto identificato dalla coppia bucket + chiave; nessun identificatore citabile a questo livello	Container avviabile rapidamente, ma immagini Docker community recenti difficili da reperire (pinnato un tag storico); uso quotidiano (console	Licenza open source (AGPL v3, edizione Community); API S3 compatibile, portabile verso altri storage compatibili con S3 o cloud

					web e client mc) immediato — verificato con creazione bucket, upload e ispezione di oggetti	
OpenMetadata	Catalogo di metadati: censisce le sorgenti, costruisce glossario e osserva il lineage	Entità tipizzate a schema fisso	Mantiene una propria cronologia delle versioni delle entità di metadati, indipendente dagli snapshot della sorgente osservata	Ogni entità identificata da un nome completamente qualificato	Richiede il docker-compose ufficiale.	Open source (Apache 2.0)
DataHub	Catalogazione e governance dei metadati; studiato solo tramite istanza demo pubblica (nessuna installazione)	Grafo di entità e aspetti componibili (Columns, Properties, Glossary, Lineage)	Versionamento degli Aspect; ogni volta che un aspetto cambia, viene creata automaticamente una nuova versione	Percorso gerarchico/URN (es. PostgreSQL → calm-pagoda-323403 – jaffle_shop → orders)	Nessuna installazione nel progetto; esplorazione della demo intuitiva	Apache 2.0 dichiarato
DigitalHub	Piattaforma Kubernetes-based per l'intero ciclo di vita di un progetto dati/AI, alternativa all'assemblaggio manuale dei componenti	Astrazione unica (Dataitem) su data lake e DB relazionale interni	Non verificabile: solo studio documentale, nessuna installazione prevista dal documento di progetto	Non chiarito dalla documentazione consultata	Non installato; dalla documentazione risulta orientato a ridurre l'attrito tramite ambienti di lavoro pronti (Jupyter/VS Code) integrati	Dichiarato open source;
DataVerse	Repository di pubblicazione per rendere un dataset permanente e citabile anche al di fuori del sistema	Un dataverse è una collezione di dataset, ognuno dei quali è descritto da metadati	Il versionamento è nativo poiché ogni pubblicazione crea una nuova versione. Quelle precedenti continuano ad essere reperibili e accessibili	Il sistema assegna ad ogni pubblicazione un DOI, ossia un identificatore che rende il dataset citabile anche all'esterno del sistema	Nessuna installazione. L'interfaccia web risulta molto intuitiva e semplice, mentre tramite API il processo è più laborioso, poiché richiesta la preparazione di un JSON	Si tratta di una piattaforma open source, dunque non vi è nessun vincolo e i dati sono esportabili.

Versioni dei sistemi

Sistema	Versioni
Minio	RELEASE.2025-09-07T16-13-09Z
PyIcerberg	0.11.1
DuckDB	1.5.5
MongoDB	8.2.12
PostgreSQL	17.0
OpenMetadata (Andrea)	1.12.0
OpenMetadata(Davide)	2.0.0
OpenMetadata (Mattia)	2.0.1
DataHub	v1.8.0rc3 (istanza demo pubblica)
DataVerse	V6.11 (istanza pubblica di prova)

Confronto OpenMetadata / DataHub

OpenMetadata e DataHub sono piattaforme dedicate alla catalogazione dei dati, che consentono di raccogliere, organizzare e consultare in un unico ambiente le informazioni relative a database, tabelle, colonne, utenti e dashboard, agevolandone così la ricerca, la comprensione e il monitoraggio. Le due piattaforme si differenziano tuttavia per il modello adottato: OpenMetadata si basa su una struttura più semplice e lineare, mentre DataHub adotta un approccio distinto, modellando l'informazione attraverso un grafo di entità e aspetti. Un'entità (ad esempio una tabella) può infatti avere diversi aspetti associati (proprietà tecniche, tag, documentazione) secondo una logica più flessibile e componibile, mentre le relazioni tra le entità danno luogo a un grafo navigabile anziché a uno schema rigido. Tale architettura conferisce a DataHub una maggiore flessibilità, particolarmente utile in contesti di grandi dimensioni.
Le due piattaforme condividono comunque alcune caratteristiche fondamentali: entrambe consentono di estendere i metadati, di collegarsi a numerose fonti dati mediante appositi connettori e di ricostruire il data lineage, ossia il percorso e le trasformazioni subite dai dati nei diversi sistemi. DataHub si distingue generalmente per funzionalità più avanzate in materia di gestione del lineage e personalizzazione del modello dei dati, mentre OpenMetadata privilegia la semplicità di configurazione e di utilizzo.

DigitalHub

Digital Hub è una piattaforma open-source, basata su Kubernetes, che copre l'intero ciclo di vita di un progetto dati o AI: dall'esplorazione iniziale fino alla pubblicazione in produzione. L'idea di fondo è mettere insieme in un unico ambiente le pratiche più moderne dello sviluppo software (DataOps, MLOps, DevOps e GitOps) così da non dover gestire a mano ogni pezzo della pipeline. Il lavoro si articola attorno a quattro funzioni principali: esplorare i dati in ambienti interattivi già pronti (Jupyter o VS Code) e collegati alla piattaforma; elaborare dati strutturati e non tramite datalake e database integrati, usando il framework che si preferisce (Python, container, ecc.); eseguire il codice sfruttando Kubernetes, che si occupa in automatico di creare immagini e gestire i servizi;

e infine integrare, cioè costruire nuovi servizi AI ed esporre i dati con standard comuni, in modo da collegarli facilmente ad altri strumenti.
Nel nostro contesto, Digital Hub non va inteso come un quarto modello di dato accanto a MongoDB, PostgreSQL e Iceberg, ma come un'architettura già assemblata dietro un'unica interfaccia. Le differenze principali riguardano tre aspetti.
La generazione dei dati: mentre nel prototipo avviene tramite l'esecuzione locale di uno script Python, su Digital Hub sarebbe gestita, versionata e loggata automaticamente dalla piattaforma stessa, all'interno dei suoi ambienti di lavoro.
Lo storage: il prototipo tiene volutamente separati e visibili i tre modelli di dato, per poterne confrontare le caratteristiche, mentre Digital Hub nasconde questa scelta dietro un'unica astrazione (il *Dataitem*), che internamente si appoggia a un data lake e a un database relazionale senza esporre all'utente quale motore sia effettivamente in uso.
L'analisi dei dati: Digital Hub integra motori come Dremio, che svolge lo stesso ruolo di DuckDB nel prototipo (un motore SQL sopra il Data Lake), con la differenza che il collegamento è già predisposto dalla piattaforma, mentre nel prototipo è stato realizzato manualmente tramite un bridge tra PyIceberg e DuckDB.

DataVerse

Dataverse è una piattaforma open source progettata per la gestione, conservazione e pubblicazione di dataset, utilizzata soprattutto nel contesto della ricerca scientifica e accademica. Permette di raccogliere all'interno di un unico ambiente i dati e le relative informazioni descrittive, chiamate metadati, come il titolo del dataset, gli autori, la descrizione, le parole chiave e la data di pubblicazione. In questo modo i dataset possono essere facilmente consultati, condivisi e riutilizzati da altri utenti.
Un aspetto importante di Dataverse è la possibilità di pubblicare e versionare i dataset. Quando un dataset viene modificato e viene pubblicata una nuova versione, Dataverse mantiene lo storico delle versioni precedenti, permettendo di identificare le diverse modifiche effettuate nel tempo. Inoltre, i dataset pubblicati possono ricevere un identificativo persistente, rendendoli più facilmente citabili e rintracciabili.
La piattaforma può essere utilizzata sia attraverso una interfaccia web, che consente all'utente di creare e pubblicare manualmente un dataset, sia attraverso delle API REST, che permettono di automatizzare le stesse operazioni tramite applicazioni o strumenti come curl. Per questo motivo Dataverse può essere considerato non tanto un database tradizionale, come MongoDB, quanto un repository dedicato alla pubblicazione, condivisione e conservazione dei dati e dei relativi metadati.

Prove pratiche

Le configurazioni dei sistemi e le prove pratiche sono reperibili al seguente link GitHub:
https://github.com/AndreaFr0/BigDataManagement_Project.git